AI-Powered Circular-Economy Marketplace

Database architecture, EERDs and document models for a microservice system with polyglot persistence

CONTENTS

Page 2 records the implemented stack. The MongoDB sections extend this document; they do not redesign or duplicate any PostgreSQL entity.

GOVERNING RULES

- Each microservice owns exactly one database. Foreign keys never cross a database boundary, and no reference ever crosses the engine boundary as a constraint.
- PostgreSQL holds structured transactional domain state. MongoDB holds messaging, comments, notifications, AI assistant history and search behaviour.
- No PostgreSQL entity is duplicated in MongoDB. MongoDB stores user_id, listing_id, offer_id, deal_id and review_id as application-level references only.
- The core business flow is Listing to Offer to Deal to Handover, followed by a Review. Everything conversational or behavioural sits alongside it, never inside it.

Auth DB

PostgreSQL

Accounts, multi-provider credentials, sessions, password resets, roles and permissions.

Marketplace DB

PostgreSQL

Vendor and corporate profiles, listings, nested categories, locations, relationships.

Transaction DB

PostgreSQL

Offers, accepted-offer to deal conversion, immutable deal terms, append-only status audit trail.

Review DB

PostgreSQL

Written reviews AND 1-5 numerical ratings, permanently persisted. Vendor replies. Aggregates.

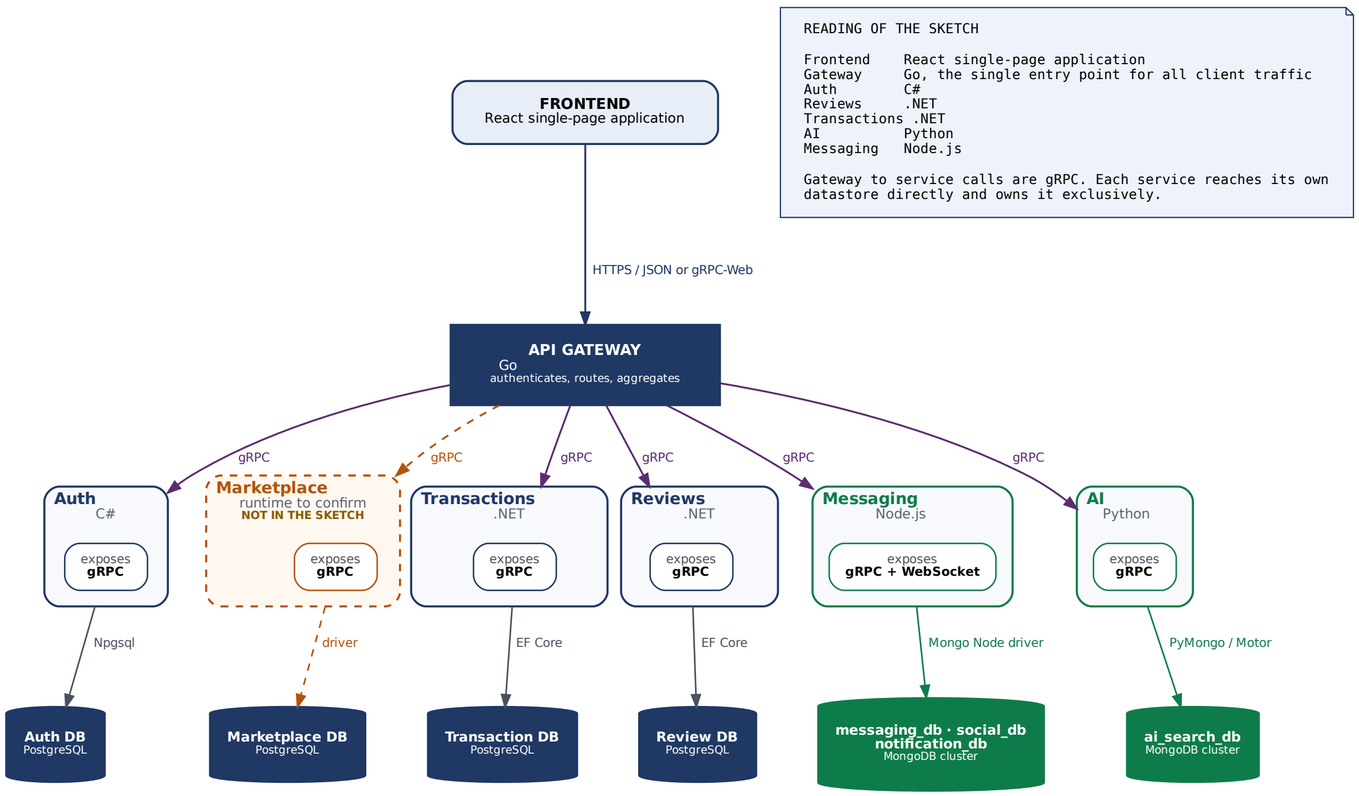
MongoDB cluster

4 logical databases

Messaging, Social, Notification and AI/Search services own nine collections between them.

TECHNOLOGY STACK & SERVICE TOPOLOGY

as implemented — polyglot services behind a Go gateway, each owning its own datastore



LOGICAL SERVICE → DEPLOYED PROCESS			
Bounded context in this document	**Deployed process**	**Datastore it owns**	**Engine**
Auth Service	Auth • C#	Auth DB	PostgreSQL
Marketplace Service	not in the sketch — to confirm	Marketplace DB	PostgreSQL
Transaction Service	Transactions • .NET	Transaction DB	PostgreSQL
Review Service	Reviews • .NET	Review DB	PostgreSQL
Messaging Service	Messaging • Node.js	messaging_db	MongoDB
Social Service	Messaging • Node.js (assumed)	social_db	MongoDB
Notification Service	Messaging • Node.js (assumed)	notification_db	MongoDB
AI Assistant & Search Service	AI • Python	ai_search_db	MongoDB

A bounded context is a data-ownership boundary; a process is a deployment unit. Several contexts may share one process, each keeping its own logical database. The rule that survives is per-database ownership: no process reads a logical database it does not own.

THREE POINTS TO CONFIRM

1 NO MARKETPLACE SERVICE IS DRAWN
LISTING, CATEGORY, LOCATION, VENDOR and CORPORATE are the centre of the domain and need an owner. Either the service exists and was left off the sketch, or those tables are currently inside Transactions (.NET), which would make one service own two bounded contexts.

2 REST BETWEEN A SERVICE AND ITS DATABASE
The sketch labels service-to-database links REST. Databases are reached over their own wire protocols through drivers - Npgsql / EF Core, the MongoDB Node and Python drivers - not over HTTP. Read as: each service exposes REST outward and talks to its store through a driver.

3 GRPC FROM A BROWSER
Browsers cannot open raw gRPC connections. The Go gateway almost certainly terminates HTTP/JSON or gRPC-Web from React and re-issues native gRPC inward. Worth stating explicitly.

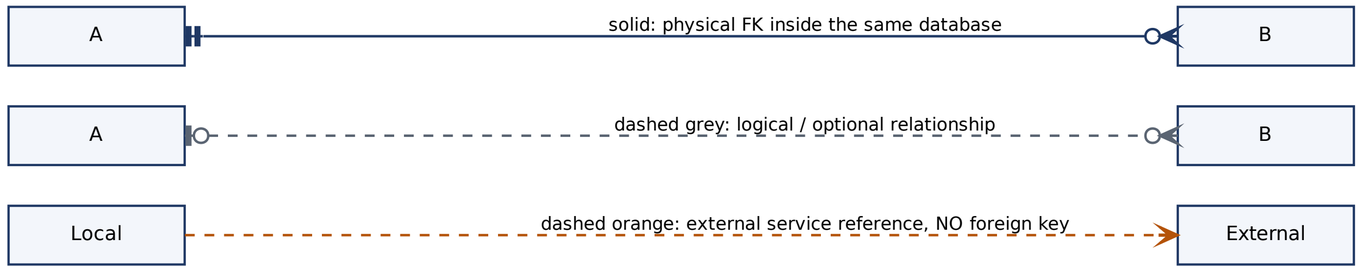
NOTATION KEY

Crow’s Foot notation used throughout • read the symbol nearest an entity as that entity’s cardinality

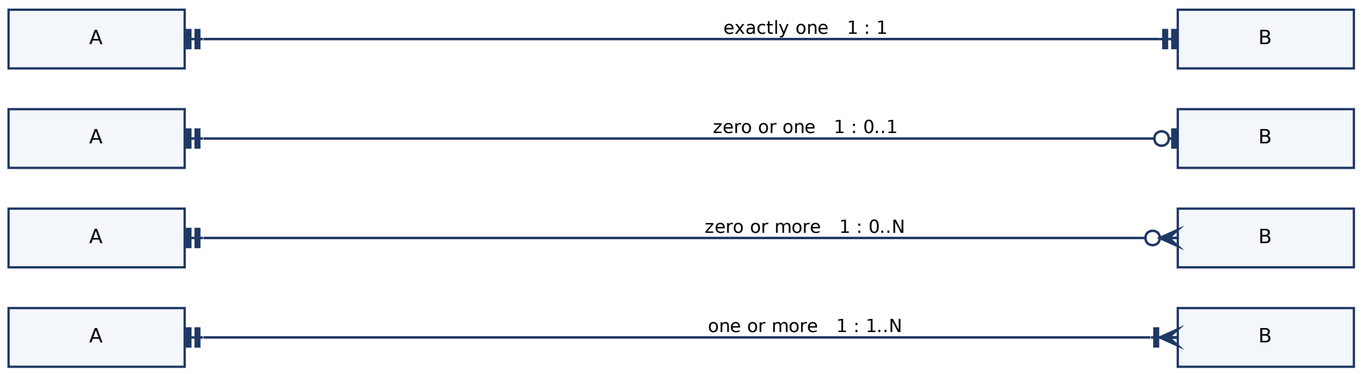
ATTRIBUTE MARKERS	
PK	Primary key (underlined)
FK	Foreign key — always within the same database
UK	Unique constraint
EXT	External service reference — plain immutable ID, **no** FK
PK,FK	Composite key of an associative entity
NULL	Nullable attribute — everything else is NOT NULL

ENTITY COLOURS	
ENTITY	Strong entity
ENTITY	Dependent entity (existence depends on its parent)
ENTITY	Associative entity resolving an M:N relationship
ENTITY	Derived / cached read model — never authoritative
SERVICE	Stub for an entity owned by another microservice

Line styles



Cardinality



EERD 1 • AUTH DB

Auth Service • PostgreSQL • owns identity, authentication and RBAC only

M:N USER ROLE resolved by USER_ROLE
M:N ROLE PERMISSION resolved by ROLE_PERMISSION
No foreign key in this database leaves this database.

USER

PK	**user_id**	UUID
UK	**email**	VARCHAR(320)
	email_verified	BOOLEAN
UK	**phone_number**	VARCHAR(20) *NULL*
	phone_verified	BOOLEAN
	status	VARCHAR(16)
	created_at	TIMESTAMPTZ
	updated_at	TIMESTAMPTZ
	last_login_at	TIMESTAMPTZ *NULL*

status: PENDING | ACTIVE | SUSPENDED | DEACTIVATED
no credentials stored here - see AUTH_IDENTITY

AUTH_IDENTITY

PK	**identity_id**	UUID
FK	**user_id**	UUID
	provider	VARCHAR(20)
	provider_user_id	VARCHAR(255)
	password_hash	VARCHAR(255) *NULL*
	created_at	TIMESTAMPTZ

UNIQUE (provider, provider_user_id)
provider: LOCAL | GOOGLE | APPLE | MICROSOFT
password_hash used only when provider = LOCAL

SESSION

PK	**session_id**	UUID
FK	**user_id**	UUID
UK	**refresh_token_hash**	VARCHAR(255)
	expires_at	TIMESTAMPTZ
	created_at	TIMESTAMPTZ
	revoked_at	TIMESTAMPTZ *NULL*

tokens stored hashed, never in plaintext

PASSWORD_RESET

PK	**reset_id**	UUID
FK	**user_id**	UUID
UK	**token_hash**	VARCHAR(255)
	expires_at	TIMESTAMPTZ
	used_at	TIMESTAMPTZ *NULL*
	created_at	TIMESTAMPTZ

single-use: used_at set on redemption

ROLE

PK	**role_id**	SMALLINT
UK	**name**	VARCHAR(30)
	description	VARCHAR(255) *NULL*

USER | VENDOR | CORPORATE | ADMIN | MODERATOR
open lookup table - further roles need data, not schema, changes

USER_ROLE
associative entity - resolves USER M:N ROLE

PK,FK	**user_id**	UUID
PK,FK	**role_id**	SMALLINT
	assigned_at	TIMESTAMPTZ

PERMISSION

PK	**permission_id**	SMALLINT
UK	**name**	VARCHAR(50)
	description	VARCHAR(255) *NULL*

CREATE_LISTING | EDIT_LISTING | DELETE_LISTING | MAKE_OFFER
MANAGE_USERS | VERIFY_VENDOR | MODERATE_CONTENT | ...

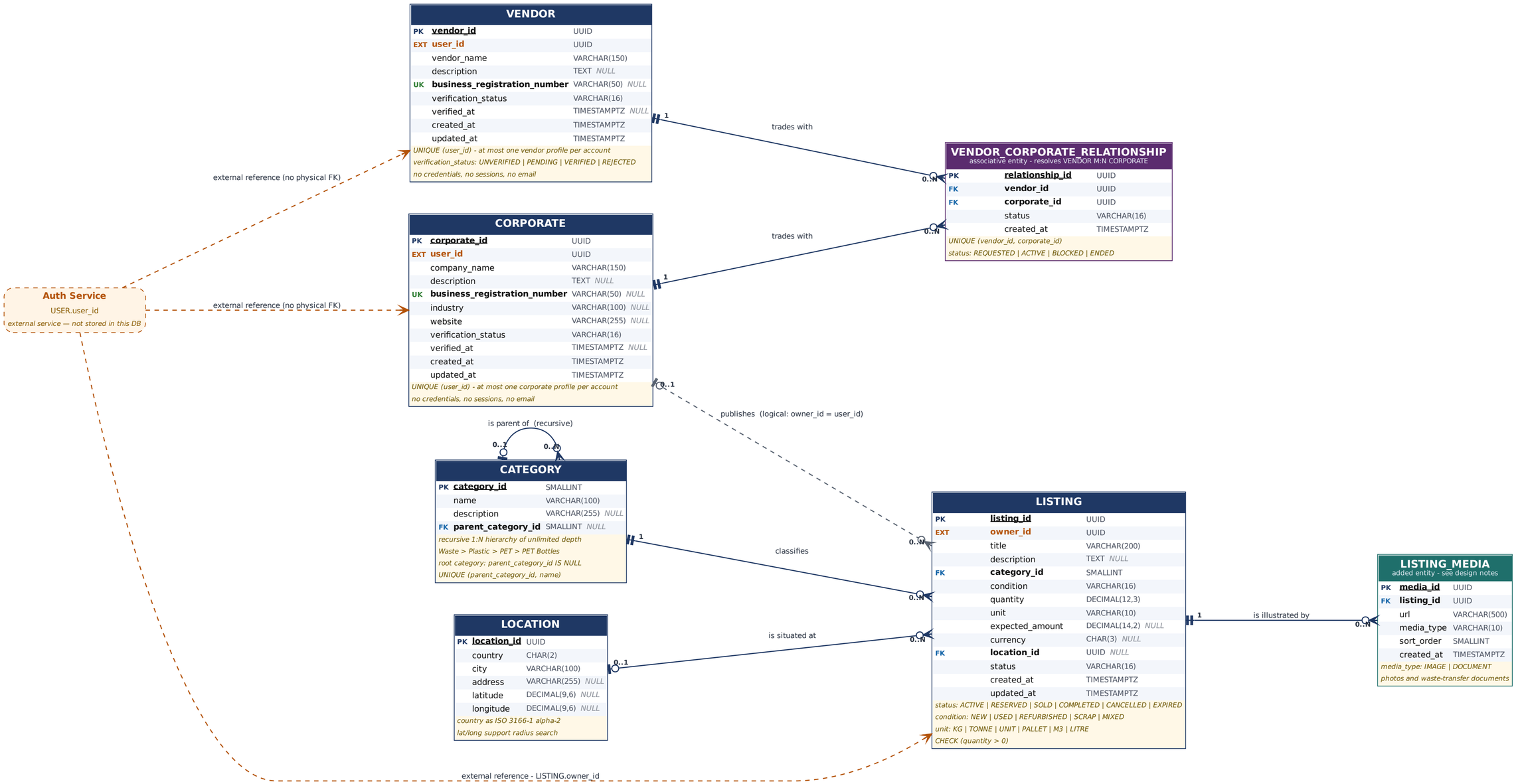
ROLE_PERMISSION
associative entity - resolves ROLE M:N PERMISSION

PK,FK	**role_id**	SMALLINT
PK,FK	**permission_id**	SMALLINT

authenticates via (USER 1 — AUTH_IDENTITY 1..N)
opens (USER 1 — SESSION 0..N)
requests (USER 1 — PASSWORD_RESET 0..N)
is assigned (USER 1 — USER_ROLE 0..N)
is granted to (ROLE 1 — USER_ROLE 0..N)
holds (ROLE 1 — ROLE_PERMISSION 0..N)
is granted through (PERMISSION 1 — ROLE_PERMISSION 0..N)

EERD 2 • MARKETPLACE DB

Marketplace Service • PostgreSQL • organisation profiles, listings, taxonomy, locations



EERD 3 • TRANSACTION DB

Transaction Service • PostgreSQL • Listing → Offer → Deal → Handover (no orders, carts or refunds)

CORE BUSINESS FLOW

LISTING (Marketplace DB)
  |
  v
OFFER
  | accepted
  v
DEAL
  |
  v
HANDOVER / COMPLETION

Deliberately NOT modelled:
ORDER, ORDER_ITEM, CART, REFUND

Marketplace Service

LISTING.listing_id

external service — not stored in this DB

external reference (no physical FK)

Marketplace Service

VENDOR.vendor_id / CORPORATE.corporate_id

external service — not stored in this DB

external reference (no physical FK)

OFFER

PK	**offer_id**	UUID
EXT	**listing_id**	UUID
EXT	**buyer_id**	UUID
EXT	**seller_id**	UUID
	offered_amount	DECIMAL(14,2)
	currency	CHAR(3)
	message	TEXT *NULL*
	status	VARCHAR(16)
	created_at	TIMESTAMPTZ
	expires_at	TIMESTAMPTZ *NULL*
	responded_at	TIMESTAMPTZ *NULL*

status: PENDING | ACCEPTED | REJECTED | WITHDRAWN | EXPIRED
buyer_id = vendor making the offer, seller_id = corporate disposing
CHECK (offered_amount > 0)

1

when ACCEPTED becomes

0..1

DEAL

PK	**deal_id**	UUID
FK,UK	**offer_id**	UUID
EXT	**listing_id**	UUID
EXT	**buyer_id**	UUID
EXT	**seller_id**	UUID
	agreed_amount	DECIMAL(14,2)
	currency	CHAR(3)
	status	VARCHAR(16)
	created_at	TIMESTAMPTZ
	completed_at	TIMESTAMPTZ *NULL*
	cancelled_at	TIMESTAMPTZ *NULL*

status: AGREED | HANDOVER_PENDING | COMPLETED | CANCELLED | DISPUTED
UNIQUE (offer_id) - an accepted offer yields at most one deal
agreed_amount and currency are immutable once written
party ids duplicated deliberately: the deal is a standalone legal record

1

logs transition

1..N

Auth Service

USER.user_id

external service — not stored in this DB

external reference - changed_by

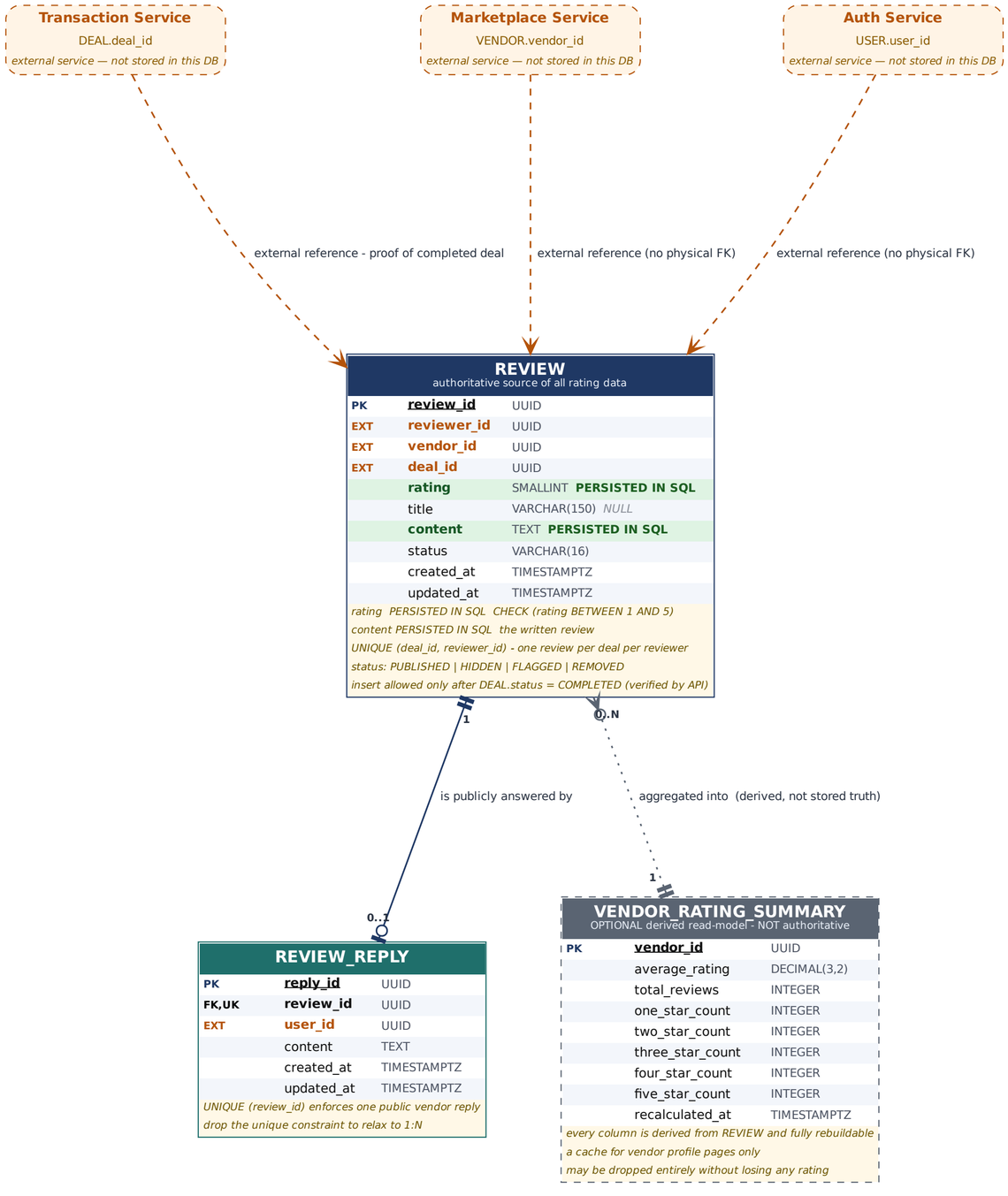
DEAL_STATUS_HISTORY

PK	**history_id**	UUID
FK	**deal_id**	UUID
	previous_status	VARCHAR(16) *NULL*
	new_status	VARCHAR(16)
EXT	**changed_by**	UUID *NULL*
	changed_at	TIMESTAMPTZ
	reason	VARCHAR(255) *NULL*

append-only audit trail - never updated or deleted
first row is NULL -> AGREED, written with the deal
changed_by NULL when the transition is system generated
AGREED -> HANDOVER_PENDING -> COMPLETED

EERD 4 • REVIEW DB

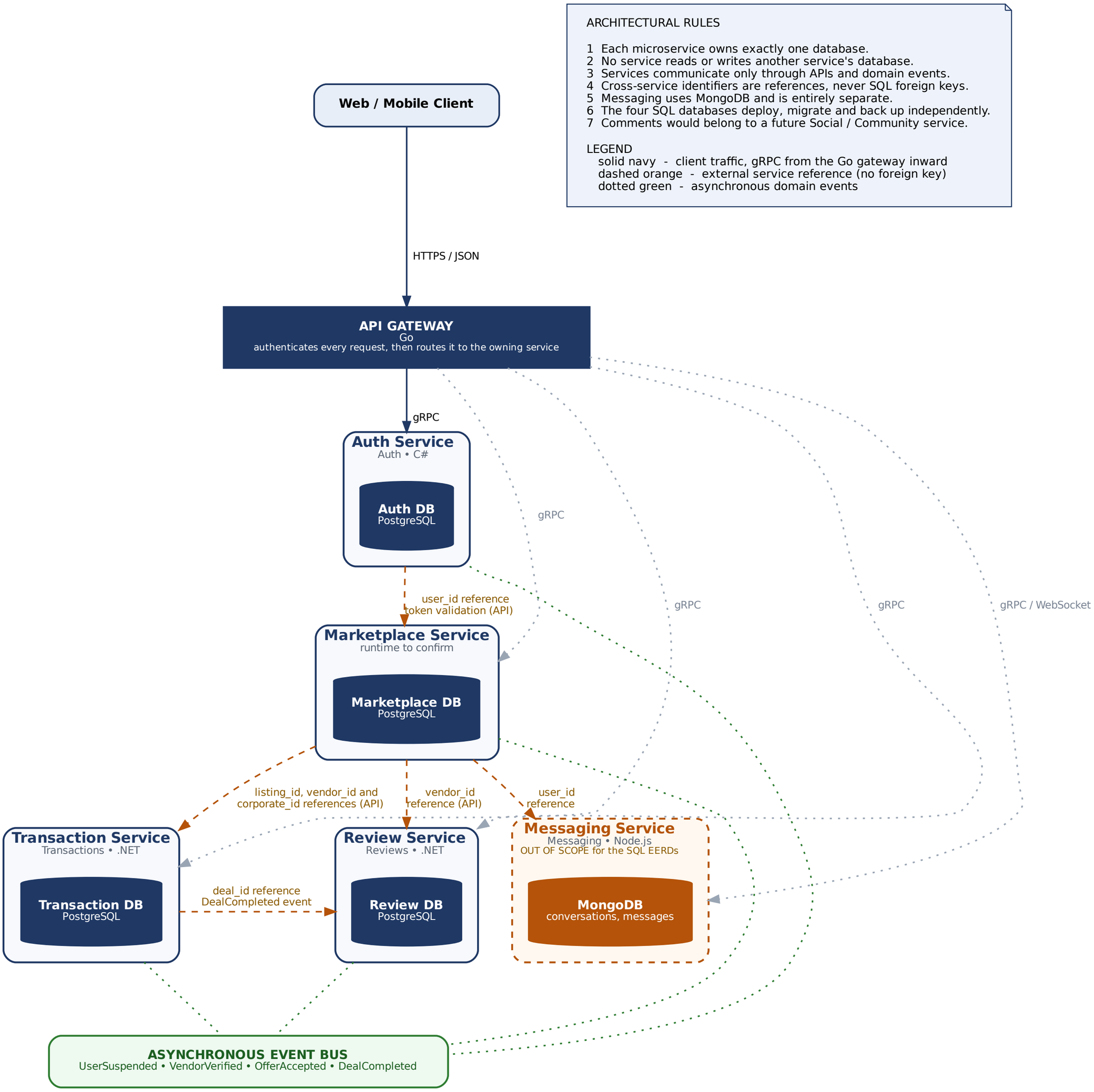
Review Service • PostgreSQL • written reviews AND numerical ratings are permanently persisted in SQL




```
PUBLIC VENDOR RATING - DERIVED, NOT STORED AS TRUTH

SELECT vendor_id,
       AVG(rating)                          AS average_rating,
       COUNT(*)                             AS total_reviews,
       COUNT(*) FILTER (WHERE rating = 1) AS one_star,
       COUNT(*) FILTER (WHERE rating = 2) AS two_star,
       COUNT(*) FILTER (WHERE rating = 3) AS three_star,
       COUNT(*) FILTER (WHERE rating = 4) AS four_star,
       COUNT(*) FILTER (WHERE rating = 5) AS five_star
FROM   review
WHERE  status = 'PUBLISHED'
GROUP  BY vendor_id;

No separate RATING entity: every review carries exactly one
rating, so a 1:1 table would add a join with no benefit.
```

DIAGRAM 5 • MICROSERVICE / DATABASE ARCHITECTURE

database-per-service • no shared database • no physical cross-database foreign keys



MONGODB DATA MODEL

polyglot persistence — why part of this system is document-oriented, and exactly which part

The four PostgreSQL databases in the preceding sections hold the system's domain state: who exists, what is listed, what was agreed and what was rated. That data is highly structured, benefits from constraints and must never be lost or silently wrong. It stays exactly where it is.

MongoDB is introduced for a different class of data: the conversational, social and behavioural record that surrounds those transactions. Messages, comments, notifications, AI assistant turns and search behaviour share a set of properties that make a document store the better fit — and that would make a rigid relational schema a recurring source of migrations.

PROPERTIES OF THE DATA MONGODB HOLDS HERE

Document-oriented

A message, a comment, a notification and an assistant turn are each naturally one self-contained document. Nothing about them wants to be spread across five normalised tables and reassembled by a join.

Semi-structured

A DEAL_COMPLETED notification carries an amount; a NEW_COMMENT carries a snippet. Chatbot metadata differs per model version. Search filters follow whatever the search UI currently offers.

High-volume

Views, clicks, messages and notifications outnumber deals by orders of magnitude. These collections will be the largest datasets in the system long before the marketplace is large.

Append-heavy

Written once, read many, almost never updated in place. There is no lock contention on a hot row, and no update anomaly to normalise away.

Conversational

Access is a time-ordered, paged stream scoped to one thread or one user — the access pattern a compound index on (parent_id, created_at) serves almost perfectly.

Event / history-oriented

This is a record of what happened, not the current state of anything. History is not corrected; it is appended to.

Expected to evolve

New notification types, new interaction types, new AI execution metadata and new attachment kinds arrive with most releases. Each must be additive, not a migration on the largest tables in the system.

WHAT MONGODB DOES NOT HOLD

No duplicated domain entities

USER, VENDOR, CORPORATE, LISTING, OFFER, DEAL, REVIEW and RATING remain owned exclusively by their PostgreSQL services. No copy of any of them exists in MongoDB. A denormalised display name is never stored; it is resolved through the owning service's API at read time.

Only references cross the boundary

MongoDB documents store user_id, listing_id, offer_id, deal_id, review_id, category_id and location_id as plain immutable UUID values. These are application-level references: no foreign key, no constraint, no cascade, no cross-engine join and no transaction spanning both engines.

Nothing that must be transactionally correct

No money, no ownership, no agreement and no rating is stored in MongoDB. If a view event is lost the system loses a little ranking signal; if a deal amount were lost the system would be wrong. That asymmetry is the entire basis for the split.

SERVICE OWNERSHIP IS PRESERVED

Four MongoDB services, four logical databases

Messaging, Social, Notification and AI Assistant & Search each own their own collections and their own logical database. They may share one physical cluster for operational economy, but no service reads another service's collections. Database-per-service is a logical rule about ownership, not a promise about how many servers are running.

A note on comments

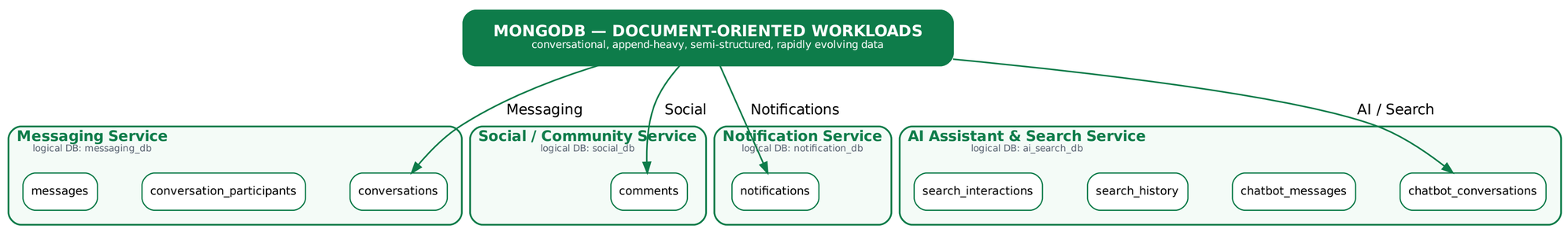
The PostgreSQL sections listed a Social/Community service as a possible future addition and deliberately kept comments out of the Transaction database. This section resolves that: comments live in MongoDB, owned by the Social Service. No change is required to any existing PostgreSQL EERD.

The rule that keeps polyglot persistence coherent

Two engines are only worth their operational cost if the boundary between them is decided by the nature of the data rather than by convenience. Anything a user is entitled to rely on — an identity, a listing, an agreed price, a rating — belongs in PostgreSQL, under constraints, in a transaction. Anything that records what happened around those facts belongs in MongoDB, where documents can differ, arrive fast and change shape between releases. Neither engine ever holds a copy of the other's data, and the dependency points one way: MongoDB stores PostgreSQL identifiers, never the reverse.

MONGODB ARCHITECTURE

four services • four logical databases • one physical cluster • no service reads another service’s collections




```
PHYSICAL CLUSTER  vs  LOGICAL OWNERSHIP

All four services may share one MongoDB replica set / Atlas cluster
for operational economy. That is a deployment decision, not an
architectural one.

Each service owns a separate logical database and a separate
database user whose role grants read/write on its own collections
only. The Social Service cannot read messages; the Notification
Service cannot read chatbot_messages.

If isolation, scaling or retention needs diverge, a logical database
can be lifted onto its own cluster with no change to the data model.
```

```
WHY MONGODB FOR THIS DATA

document-oriented    a message, a comment and a notification are each
                     naturally one self-contained document
semi-structured      notification bodies, chatbot metadata and search
                     filters differ per document
high volume          messages, views and clicks vastly outnumber deals
append-heavy         written once, read many, rarely updated in place
conversational       ordered, paged, time-series-like access patterns
event / history      an immutable record of what happened, not state
fast-evolving        new notification types, new AI metadata and new
                     interaction types must not require a migration
```

NOTATION KEY • MONGODB DOCUMENT SCHEMAS

document-schema notation — deliberately different from the Crow’s Foot EERD notation used for the PostgreSQL databases

example_collection collection
Owning Service • logical database

_id	**_id**	ObjectId	document identifier
REF	**conversation_id**	ObjectId	reference to another collection
EXT	**user_id**	UUID	reference to a PostgreSQL service
	content	String	
	created_at	Date	

metadata { } nested object — flexible, schema-light

	model	String | null
	tokens_used	Number | null

attachments [ ] array of sub-documents — 0..N embedded

	url	String
	type	String

amber italic rows carry constraints, enum values and design notes

FIELD MARKERS	
_id	Document identifier (ObjectId), unique within the collection
REF	Reference to a document in another MongoDB collection (ObjectId)
EXT	Application-level reference to an entity owned by a PostgreSQL service. **Never** a foreign key. No join, no constraint, no cascade.
STRUCTURE	
{ }	Nested object embedded in the parent document
[ ]	Array of embedded sub-documents (bounded, small cardinality)
LINES	
solid green	Reference between two collections inside the same MongoDB cluster
dashed orange	Application-level reference to a PostgreSQL service

```
EMBED  or  REFERENCE ?

EMBEDDED when the data is bounded, always read with the parent
and written at the same time:  attachments, reactions, participants,
filters, metadata, last_message.

REFERENCED when the child set is unbounded or independently paged:
messages, chatbot_messages, comments, search_interactions.

A conversation may accumulate tens of thousands of messages, so
messages are never embedded in the conversation document. The 16 MB
BSON document limit and the cost of rewriting a growing document
both argue against it.
```

MESSAGING DATA MODEL

Messaging Service • MongoDB • messaging_db — user-to-user conversations, participants and messages

```
TYPICAL READ PATTERNS

1  Inbox        conversations where participants.user_id = me,
                sorted by updated_at desc
2  Thread       messages where conversation_id = C,
                sorted by created_at desc, paged
3  Unread       conversation_participants for me, compared
                against the newest message in each conversation
```

Auth Service
USER.user_id
PostgreSQL — owned by another service

Marketplace Service
LISTING.listing_id
PostgreSQL — owned by another service

sender_id

participants[].user_id

listing_id

replies to (self-reference)

0..1 — 0..N

messages collection
Messaging Service • messaging_db

_id	**_id**	ObjectId	creation time is embedded in the ObjectId
REF	**conversation_id**	ObjectId	
EXT	**sender_id**	UUID	Auth Service
	content	String	
	message_type	String	text | image | file | system

attachments [ ] array of sub-documents 0..N

url	String	object-store key
type	String	
name	String	
size	Number	bytes

REF	**reply_to_message_id**	ObjectId | null	same collection

reactions [ ] array of sub-documents 0..N, bounded in practice

EXT	**user_id**	UUID	
	reaction	String	emoji shortcode

created_at	Date	
updated_at	Date	set when edited
deleted_at	Date | null	soft delete

stored in their own collection, never embedded: a single conversation may hold tens of thousands of messages and would breach the 16 MB document limit
reply_to_message_id is a self-reference supporting quoted replies

conversations collection
Messaging Service • messaging_db

_id	**_id**	ObjectId	

participants [ ] array of sub-documents bounded: 2 for a direct chat

EXT	**user_id**	UUID	Auth Service
	role	String	vendor | corporate

EXT	**listing_id**	UUID | null	Marketplace Service
	created_at	Date	
	updated_at	Date	sorted on for the inbox

last_message { } nested object denormalised for the inbox list

REF	**message_id**	ObjectId	
EXT	**sender_id**	UUID	
	content_preview	String	truncated
	sent_at	Date	

listing_id is null for a general conversation and set when the chat starts from a listing: Vendor sees 500 kg PET Bottles and clicks Message Seller
last_message is a deliberate denormalisation: it renders the whole inbox without a second query. The messages collection remains the source of truth.

1 — contains — 0..N

1 — has participant state — 2..N

conversation_participants collection
Messaging Service • messaging_db

_id	**_id**	ObjectId	
REF	**conversation_id**	ObjectId	
EXT	**user_id**	UUID	Auth Service
	joined_at	Date	
REF	**last_read_message_id**	ObjectId | null	
	last_read_at	Date | null	
	muted	Boolean	default false
	archived	Boolean	default false

per-participant state kept out of the conversation document so that one user muting or archiving does not rewrite a document the other user reads
unread count = messages after last_read_message_id in this conversation
unique (conversation_id, user_id)

SOCIAL / COMMENTS DATA MODEL

Social Service • MongoDB • social_db — threaded comments on marketplace listings

WHY MONGODB RATHER THAN A FIFTH POSTGRESQL DATABASE

Comments are unbounded, append-heavy, edited rarely and read in tree order. Recursive CTEs in SQL would work, but the collection is also where reactions, mentions, media and moderation flags will land next, and each of those is an additive document change here and a migration in a relational table.

The earlier PostgreSQL EERDs listed a Social/Community service as a future option. This section replaces that option: comments live in MongoDB, owned by the Social Service.

THREADING EXAMPLE — one listing

Comment A *parent_comment_id: null*

- ├─ Reply A1 *parent_comment_id: A*
- │ └─ Reply A1a *parent_comment_id: A1*
- └─ Reply A2 *parent_comment_id: A*

Comment B *parent_comment_id: null*

Auth Service
USER.user_id
PostgreSQL — owned by another service

Marketplace Service
LISTING.listing_id
PostgreSQL — owned by another service

user_id and reactions[].user_id

listing_id

is parent of (recursive)

0..1 — 0..N

comments collection
Social Service • social_db

_id	**_id**	ObjectId	
EXT	**listing_id**	UUID	Marketplace Service
EXT	**user_id**	UUID	Auth Service, the author
	content	String	
REF	**parent_comment_id**	ObjectId | null	self-reference, null = top level

reactions [ ] array of sub-documents 0..N embedded

EXT	**user_id**	UUID	
	reaction	String	like | helpful | ...

	created_at	Date	
	updated_at	Date	
	deleted_at	Date | null	soft delete keeps the thread intact

parent_comment_id gives unlimited threading depth without a schema change
a deleted parent is tombstoned rather than removed, so replies keep their place
comments are NOT in the Transaction PostgreSQL database and NOT in any SQL EERD

NOTIFICATION DATA MODEL

Notification Service • MongoDB • notification_db — heterogeneous, per-user, append-heavy

```
WHY A DOCUMENT STORE SUITS NOTIFICATIONS

Notification documents are heterogeneous. A DEAL_COMPLETED carries a
deal id and an amount; a NEW_COMMENT carries a comment id and a
snippet; a NEW_REVIEW carries a rating. In a relational table this
becomes either a wide sparse row, a set of nullable columns per type,
or an EAV side table.

Here the shared fields are fixed and the variable part sits in
entity{} and in the rendered title/body. Adding a NEW_MATCH type when
the AI matching engine ships is a producer change and a client
change - no schema migration, no downtime, no backfill of old rows.

The volume argument is the same: notifications are written once per
event per recipient, read in a single user-scoped feed, and rarely
updated after is_read flips to true.
```

EVENT SOURCES — consumed from the event bus

Producing service	Domain event	notification.type
Messaging (MongoDB)	MessageSent	NEW_MESSAGE
Social (MongoDB)	CommentCreated	NEW_COMMENT, COMMENT_REPLY
Transaction (PostgreSQL)	OfferCreated	NEW_OFFER
Transaction (PostgreSQL)	OfferAccepted	OFFER_ACCEPTED
Transaction (PostgreSQL)	DealCompleted	DEAL_COMPLETED
Review (PostgreSQL)	ReviewPublished	NEW_REVIEW

Auth Service
USER.user_id
PostgreSQL — owned by another service

user_id_and_actor_id

notifications collection
Notification Service • notification_db

_id	**_id**	ObjectId	
EXT	**user_id**	UUID	recipient, Auth Service
	type	String	see enumeration below
	title	String	
	body	String	
EXT	**actor_id**	UUID | null	who caused it, null if system
	entity	{ } nested object	polymorphic target
	type	String	listing | offer | deal | review | comment | conversation
EXT	**id**	String	id of the referenced entity
	is_read	Boolean	default false
	created_at	Date	
	read_at	Date | null	

type: NEW_MESSAGE | NEW_COMMENT | COMMENT_REPLY | NEW_OFFER | OFFER_ACCEPTED | DEAL_COMPLETED | NEW_REVIEW (open set)
entity is polymorphic: entity.type names the owning service and entity.id is resolved through that service's API when the user taps it

AI CHATBOT HISTORY DATA MODEL

AI Assistant Service • MongoDB • ai_search_db — persistent, resumable assistant sessions

WHY METADATA IS DELIBERATELY UNSTRUCTURED

The assistant will change far faster than the marketplace domain. Model names, sampling parameters, tool calls, guardrail verdicts, embedding versions and RAG citations all belong to the execution of a single turn and all change with every model upgrade.

Storing them in a nested metadata object means an upgrade adds keys to new documents and leaves old documents valid. The same change in a relational table is an ALTER TABLE plus a backfill, repeated every release, on a table that is one of the largest in the system.

What is NOT stored here: no credentials, no API keys, no prompt templates that constitute intellectual property.

Auth Service
USER.user_id
PostgreSQL — owned by another service

Marketplace Service
LISTING.listing_id
PostgreSQL — owned by another service

user_id

context.listing_id

chatbot_conversations collection
AI Assistant Service • ai_search_db

_id	**_id**	ObjectId	
EXT	**user_id**	UUID	owner, Auth Service
	title	String | null	auto-generated from the first turn
	context	{ } nested object	what the session is about
EXT	**listing_id**	UUID | null	session opened from a listing
	session_type	String	extensible, see below
	created_at	Date	
	updated_at	Date	sorted on for the session list
	last_message_preview	String | null	denormalised for the list

session_type: recycling_assistant | waste_classification | product_advice | general_assistant (open set)
history is persistent so a user can reopen an earlier assistant session

1 — contains turn — 0..N

chatbot_messages collection
AI Assistant Service • ai_search_db

_id	**_id**	ObjectId	
REF	**conversation_id**	ObjectId	chatbot_conversations
EXT	**user_id**	UUID	denormalised for per-user access control
	role	String	user | assistant | system
	content	String	
	attachments	[ ] array of sub-documents	0..N, e.g. a photo of waste to classify
	type	String	image | document
	url	String	
	metadata	{ } nested object	intentionally open-ended
	model	String | null	
	temperature	Number | null	
	tokens_used	Number | null	
	latency_ms	Number | null	
	sources	[ ] array of sub-documents	RAG citations
	document_id	String	
	title	String	
	created_at	Date	

one document per turn, never one huge document per conversation: turns are appended continuously and read in pages, and a whole session would eventually approach the 16 MB BSON limit
metadata is internal telemetry and is not returned to end users

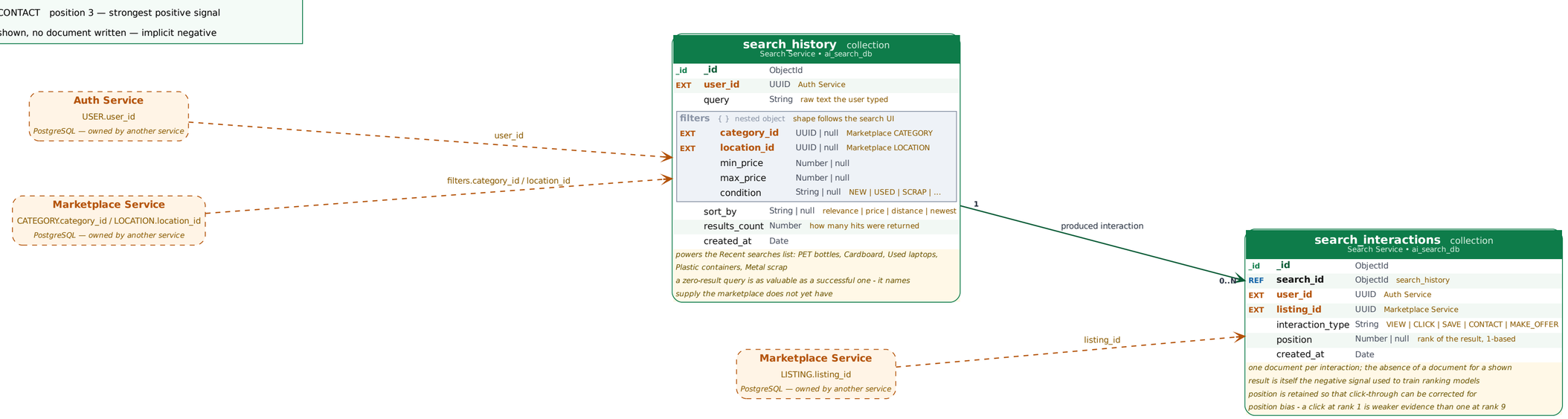
SEARCH HISTORY & SEARCH INTERACTION DATA MODEL

Search Service • MongoDB • ai_search_db — what users looked for, and what they did with the results

```
WHAT THIS DATA LATER SUPPORTS

personalised search      re-rank for the materials this user actually pursues
recommendations          you searched cardboard, these corporates post it weekly
ranking models           interaction_type as a graded relevance label,
                         corrected for position bias
behaviour analytics      funnel from VIEW to CONTACT to MAKE_OFFER
marketplace optimisation queries with many results but no CONTACT reveal
                         poor listing quality; queries with zero results
                         reveal unmet demand for the AI matching engine

This is behavioural telemetry, not domain state. It never becomes the
source of truth for anything a user is entitled to rely on.
```

WORKED EXAMPLE

search_history	query: “plastic bottles”, filters.category_id: PET, results_count: 24
├─ Listing A	VIEW position 1
├─ Listing B	VIEW position 2
├─ Listing C	CONTACT position 3 — strongest positive signal
└─ Listing D	shown, no document written — implicit negative



MONGODB COLLECTION SUMMARY

nine collections, four owning services, one cluster

Collection	Owning service · logical DB	Purpose	Growth	Primary access pattern
conversations	Messaging · messaging_db	User-to-user conversations, optionally anchored to a listing. Carries a denormalised last_message for the inbox.	one per chat	by participant, sorted by updated_at
conversation_participants	Messaging · messaging_db	Per-participant state: read position, mute, archive. Kept out of the conversation document so one user's action does not rewrite the other's.	participants × chats	by conversation, by user
messages	Messaging · messaging_db	Individual messages with attachments, reactions and quoted replies. Never embedded in the conversation.	very high	by conversation, newest first
comments	Social · social_db	Comments on marketplace listings with unlimited threading through parent_comment_id, plus embedded reactions.	high	by listing, then by thread
notifications	Notification · notification_db	Per-recipient notifications generated from domain events, with a polymorphic entity{} target and a read flag.	very high	by user, unread first
chatbot_conversations	AI Assistant · ai_search_db	AI assistant sessions with a context object and a preview, so a user can reopen an earlier conversation.	one per session	by user, sorted by updated_at
chatbot_messages	AI Assistant · ai_search_db	One document per assistant turn, including open-ended execution metadata and RAG citations.	very high	by conversation, oldest first
search_history	Search · ai_search_db	What the user searched for, with the filters and sort applied and how many results came back.	high	by user, newest first
search_interactions	Search · ai_search_db	What the user did with each result: VIEW, CLICK, SAVE, CONTACT, MAKE_OFFER, and at which rank.	highest	by search, by user, by listing

Embed or reference?

Embedded where the child set is bounded and always read with its parent: participants, attachments, reactions, filters, metadata, last_message, entity. Referenced where the child set is unbounded or independently paged: messages, chatbot_messages, comments, search_interactions. A conversation may accumulate tens of thousands of messages, so embedding them would breach the 16 MB BSON document limit and force a rewrite of a continuously growing document.

Why nine collections and not one

Each collection has a distinct owner, a distinct growth curve, a distinct retention policy and a distinct access pattern. Merging them would force one index strategy and one retention rule onto data that needs four of each, and would hand every service read access to data it has no business seeing.

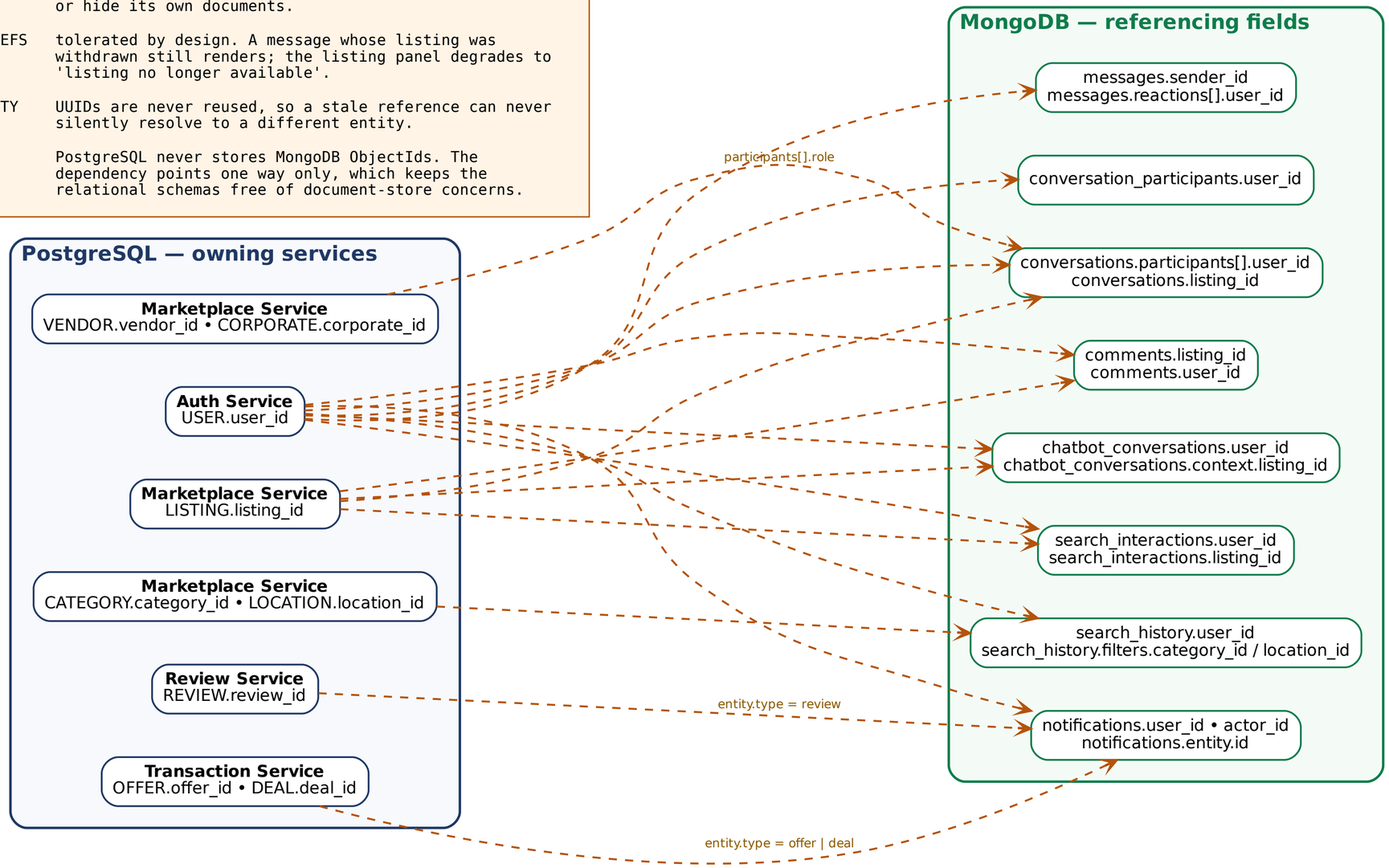
WHERE EACH FUNCTIONAL AREA LIVES

Functional area	Engine	Owning service	Entities / collections
Authentication, identity, RBAC	PostgreSQL	Auth Service	USER, AUTH_IDENTITY, SESSION, PASSWORD_RESET, ROLE, PERMISSION, USER_ROLE, ROLE_PERMISSION
Marketplace catalogue and profiles	PostgreSQL	Marketplace Service	VENDOR, CORPORATE, LISTING, CATEGORY, LOCATION, VENDOR_CORPORATE_RELATIONSHIP, LISTING_MEDIA
Offers, deals and their lifecycle	PostgreSQL	Transaction Service	OFFER, DEAL, DEAL_STATUS_HISTORY
Reviews and numerical ratings	PostgreSQL	Review Service	REVIEW, REVIEW_REPLY (+ optional derived rating summary)
User-to-user messaging	MongoDB	Messaging Service	conversations, conversation_participants, messages
Comments on listings	MongoDB	Social Service	comments
Notifications	MongoDB	Notification Service	notifications
AI assistant history	MongoDB	AI Assistant Service	chatbot_conversations, chatbot_messages
Search history and behaviour	MongoDB	Search Service	search_history, search_interactions

CROSS-SERVICE REFERENCES

every identifier MongoDB stores about another service is an application-level reference — never a foreign key

```
WHAT AN APPLICATION-LEVEL REFERENCE MEANS IN PRACTICE

The reference is a plain immutable UUID stored as a field value.
There is no $lookup across engines, no constraint, no cascade and no
transaction spanning PostgreSQL and MongoDB.

RESOLUTION      the client or a BFF layer asks the owning service's API
                for the display data - name, title, avatar - and joins
                in the response, not in the database.

INTEGRITY       maintained by domain events. VendorDeleted or
                UserSuspended cause the consuming service to tombstone
                or hide its own documents.

DANGLING REFS   tolerated by design. A message whose listing was
                withdrawn still renders; the listing panel degrades to
                'listing no longer available'.

IMMUTABILITY    UUIDs are never reused, so a stale reference can never
                silently resolve to a different entity.

DIRECTION       PostgreSQL never stores MongoDB ObjectIds. The
                dependency points one way only, which keeps the
                relational schemas free of document-store concerns.
```



MONGODB INDEXING STRATEGY

indexes follow observed query patterns — every index is paid for on every write

Collection	Index	Query it serves
conversations	{ participants.user_id: 1, updated_at: -1 }	The inbox: every conversation I am in, newest activity first. A multikey index over the participants array.
	{ listing_id: 1 }	Conversations started from a given listing. Sparse: most conversations have a null listing_id.
conversation_participants	{ conversation_id: 1, user_id: 1 } unique	Fetch or update my state in one conversation; the unique constraint prevents a duplicate participant row.
	{ user_id: 1, archived: 1 }	My inbox filtered by archived state, and the source of unread counts.
messages	{ conversation_id: 1, created_at: -1 }	The single most important index in the system: one thread, newest first, paged. Serves the thread view and the unread count.
	{ sender_id: 1, created_at: -1 }	A user's own message history, used for moderation and for account export.
comments	{ listing_id: 1, created_at: -1 }	All comments on a listing, newest first.
	{ parent_comment_id: 1, created_at: 1 }	Replies under one comment, in thread order. Sparse: null for top-level comments.
	{ user_id: 1, created_at: -1 }	A user's own comment history, for moderation and account export.
notifications	{ user_id: 1, created_at: -1 }	The notification feed for one user, newest first.
	{ user_id: 1, is_read: 1 }	The unread badge count. A partial index on is_read: false keeps it small, since most notifications end up read.
chatbot_conversations	{ user_id: 1, updated_at: -1 }	My assistant sessions, most recently used first.
chatbot_messages	{ conversation_id: 1, created_at: 1 }	One session replayed in order — ascending here, unlike messages, because an assistant transcript is read from the top.
	{ user_id: 1, created_at: -1 }	Per-user access control and account export.
search_history	{ user_id: 1, created_at: -1 }	The Recent searches list.
	{ query: "text" } or { query: 1 }	Aggregate query analysis: what is being searched for, and what returns nothing. Analytics only — not on the user read path.
search_interactions	{ search_id: 1 }	All interactions belonging to one search, for building training rows.
	{ user_id: 1, created_at: -1 }	One user's behavioural history, for personalisation and for deletion on request.
	{ listing_id: 1, created_at: -1 }	Engagement for one listing: how often it was shown, opened and contacted.

Do not over-index

Every secondary index is written on every insert and consumes RAM in the working set. On append-heavy collections such as messages, notifications and search_interactions — which take almost all of the system's writes — a speculative index is a permanent tax paid for a query nobody runs. Index what the application actually queries, verify with explain() that the index is used, and remove indexes that $indexStats shows are never touched.

Two things that come for free

ObjectId already encodes creation time, so sorting by _id is chronological and needs no separate index on created_at alone. Compound index prefixes are reusable: { conversation_id: 1, created_at: -1 } also serves a lookup by conversation_id alone, so a second single-field index would be redundant. Index order matters — equality field first, range or sort field second.

How this list should actually be arrived at

The indexes above are the ones the documented access patterns require, and they are a starting point rather than a conclusion. The honest process is to ship the access patterns, capture real queries with the profiler, read the winning plan from explain("executionStats") and add an index only when a collection scan or an in-memory sort shows up on a query that matters. Two symptoms are worth watching in this system specifically: an unbounded sort on the messages thread view, which means the compound index is not being used in the right direction, and an index working set on search_interactions that outgrows available RAM, which is the signal to age the raw documents out rather than to add hardware.

DATA RETENTION CONSIDERATIONS

architectural options, not legal requirements — the applicable rules for a given deployment must be established separately

MongoDB holds the collections that grow fastest and that contain the most user-generated content. Retention is therefore an architectural decision with real consequences for storage cost, index size, privacy exposure and the usefulness of the product. The positions below are design options with their trade-offs stated; none of them is presented as a compliance obligation.

Messages — retain for the life of the account

A conversation is the record of how a deal was negotiated, so users expect it to still be there months later. Deletion by a user sets deleted_at rather than removing the document, which keeps reply chains and read positions intact. When an account is closed, messages can be tombstoned so the other participant's side of the conversation still renders.

Chatbot history — retain, and let the user clear it

The requirement is explicit: users must be able to return to previous AI conversations, so history is persistent by default. Deleting a chatbot_conversation should cascade to its chatbot_messages in the application layer, since MongoDB will not do it for you. Execution metadata may be worth keeping in aggregate after the content is gone.

Search history — configurable, user-clearable

The most reasonable default is a rolling window — 90 or 180 days — with a visible Clear search history control. This is the collection where the gap between product value and privacy exposure is widest, and a TTL index on created_at implements the window with no batch job.

Notifications — expire or archive

A read notification older than a few months has almost no value and is pure index weight. A TTL index on created_at, or a partial TTL that only expires documents where is_read is true, keeps the feed collection small. The underlying event remains recoverable from the owning service.

Search interactions — age out or aggregate

This is the highest-volume collection in the system. Raw interaction documents are only needed while they are recent enough to train on; older data is better held as pre-aggregated counters per listing and per query. Aggregation also removes the link back to an individual user.

Comments — soft delete, retain the thread

Hard-deleting a comment with replies orphans the replies. Setting deleted_at and rendering a tombstone preserves the shape of the discussion while removing the content.

MECHANISMS

Mechanism	Where it applies	Trade-off
Soft delete	deleted_at: Date | null on messages, comments and chatbot content	Preserves thread structure and read positions. Requires every read path to filter on it — a partial index or a view keeps that honest.
TTL index	notifications, search_history, search_interactions	MongoDB expires documents automatically on a date field. Cheap, but deletion is best-effort and runs on a background sweep, so it is not a guarantee of prompt removal.
Aggregate then discard	search_interactions, chatbot metadata	Keep the statistical value, drop the per-user detail. Reduces both storage and privacy exposure at the same time.
Archive tier	messages, chatbot_messages	Move cold documents to cheaper storage rather than deleting them, keeping them retrievable on request but out of the working set.
Account closure	all collections holding user content	Needs a defined, service-by-service procedure: which documents are deleted, which are tombstoned and which are anonymised. Because MongoDB has no cascade, this is application logic that must be written deliberately.

A caution about this section

Nothing above should be read as a legal position. Retention periods, deletion guarantees and the handling of an account-closure request are determined by the jurisdictions a deployment operates in and by the platform's own terms, and those must be established with people qualified to advise on them. What this section does claim is narrower and purely architectural: the data model must make each of these outcomes reachable. That means a date field on every collection worth expiring, a soft-delete field wherever removing a document would break a structure someone else can see, and a documented owner for every collection so there is a service responsible for carrying out a deletion request rather than a shared database nobody owns.

SECURITY & PRIVACY CONSIDERATIONS

these collections hold the most sensitive user-generated content in the system

The PostgreSQL databases hold business facts. The MongoDB collections hold what people wrote, asked and looked for — private messages, questions put to an AI assistant, and a behavioural record of everything a user searched. A leak here is materially worse than a leak of listing data, and the controls should reflect that.

Access is mediated by the owning service

No client ever talks to MongoDB. Each service exposes an API that applies its own authorisation rules, and each service authenticates to the cluster as a distinct database user whose role grants read and write on its own logical database only. The Social Service holds no credential that can read messages.

Conversations: participation is the authorisation

Every read of a conversation, its participants or its messages is filtered by the caller's own user_id against participants.user_id. A conversation_id alone must never be sufficient to read a thread — identifiers are not secrets.

Chatbot history is strictly private

A user retrieves only their own sessions. This is why user_id is denormalised onto chatbot_messages as well as onto the conversation: the ownership check can be enforced on the message query itself rather than depending on a prior lookup.

Search history is private by default

It is visible to its owner and used for that user's personalisation. Any wider use — trending materials, demand analysis, ranking models — works on aggregated or anonymised data, with the user_id dropped at the aggregation step rather than carried into the analytics store.

Messages are not globally queryable

There is no cross-collection search over message content available to other services. Moderation and support tooling that genuinely needs it should be a separate, audited, role-restricted path — not an incidental capability of a shared connection string.

AI metadata stays internal

model, temperature, tokens_used, latency_ms and RAG document_ids are operational telemetry. They are useful for cost and quality monitoring and should not be surfaced in the client API. Prompt templates and credentials are never written to the collection at all.

Attachments are references, not payloads

Attachment documents hold an object-store key, not the file. Authorisation is enforced when the download URL is issued — typically as a short-lived signed URL — so an attachment link that leaks does not stay usable.

Encryption and auditing

TLS in transit and encryption at rest are the baseline. Field-level encryption is worth considering for message content and chatbot content specifically. Administrative access to the cluster should be audited separately from application access.

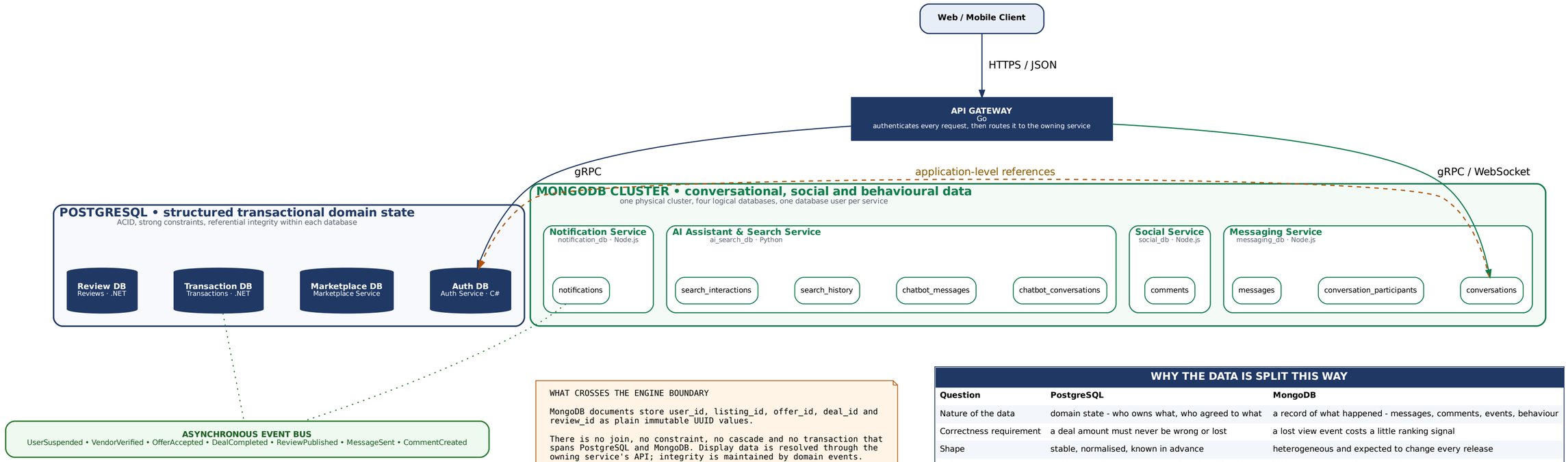
The principle underneath all of the above

Sharing one MongoDB cluster is an operational convenience. It must never become an authorisation shortcut. Every collection has exactly one owning service, that service is the only component that reads or writes it, and every other component asks through an API that can enforce a rule. The moment a second service is given a connection string to another service's collections, the architecture has quietly become a shared database with all of the coupling that the database-per-service design was adopted to avoid.

Collection	Owning service	Who may read	How the rule is enforced
conversations, conversation_participants, messages	Messaging Service	A participant of that conversation	Filter every query by the caller's user_id against participants.user_id
comments	Social Service	Anyone who can see the listing; author for edit	Public read, authenticated write; moderation is a separate audited role
notifications	Notification Service	The recipient only	user_id is the partition key of the feed; never queryable across users from the API
chatbot_conversations, chatbot_messages	AI Assistant Service	The owning user only	user_id denormalised onto messages so ownership is checked on the message query itself
search_history	Search Service	The owning user only	Wider analytics use aggregated or anonymised data with user_id dropped at aggregation
search_interactions	Search Service	No end user directly	Written by the platform, read by ranking and analytics jobs, deleted with the account

COMPLETE DATABASE ARCHITECTURE

polyglot persistence • PostgreSQL for structured transactional domain state • MongoDB for conversational, social and behavioural data




```
WHAT CROSSES THE ENGINE BOUNDARY

MongoDB documents store user_id, listing_id, offer_id, deal_id and
review_id as plain immutable UUID values.

There is no join, no constraint, no cascade and no transaction that
spans PostgreSQL and MongoDB. Display data is resolved through the
owning service's API; integrity is maintained by domain events.

The dependency points one way only: PostgreSQL never stores a
MongoDB ObjectId.
```

WHY THE DATA IS SPLIT THIS WAY

Question	PostgreSQL	MongoDB
Nature of the data	domain state - who owns what, who agreed to what	a record of what happened - messages, comments, events, behaviour
Correctness requirement	a deal amount must never be wrong or lost	a lost view event costs a little ranking signal
Shape	stable, normalised, known in advance	heterogeneous and expected to change every release
Write pattern	moderate volume, transactional, updated in place	high volume, append-only, rarely updated
Read pattern	joins across entities, aggregates, constraints	one user-scoped or thread-scoped stream, paged by time
Cost of a migration	acceptable - schema changes are rare and reviewed	prohibitive - the collections are the largest in the system